

İSG BİLDİRİM SİSTEMİ

Sprint 1 Raporu

Temel Altyapı & Şema İskeleti

Sprint:	Sprint 1 — Temel Altyapı & Şema İskeleti
Kilometre taşı:	M1 — Temel Altyapı & Kimlik Doğrulama
Zaman:	Gün 1–2 (17 iş günü planı)
Kapsam:	Üretilen dosyalar, ilişkiler ve topolojik ER haritası
Cobweb büyüklüğü:	9 düğüm / 7 kenar (kümülatif)
Belge:	Sprint 1/8

1. Sprint 1 Amacı ve Kapsamı

Projenin çalışabilir iskeletini kurmak: Docker ile veri/depolama servislerini ayağa kaldırmak, Prisma veri modelinin temelini atmak, NestJS backend ve React+Vite istemci iskeletlerini oluşturmak. Bu sprint sonunda henüz iş mantığı yoktur; ancak iki proje de derlenip ayağa kalkar ve veritabanına bağlanır.

2. Bu Sprintte Üretilen Dosyalar

Aşağıdaki tablo, bu sprintte oluşturulan her dosyayı (veya grup klasörünü) tekil olarak ele alır: hangi katmana ait olduğu, hangi modül/özellik hitap ettiği ve başlıca ilişkileri. Grup halindeki dosyalar, ilgili oldukları modül altında birlikte açıklanır.

Dosya / Grup	Katman	Modül / Özellik (ne işe yarar)	Başlıca İlişki
docker-compose.yml	Altyapı	DevOps orkestrasyonu — PostgreSQL 16, pgAdmin ve MinIO servislerini healthcheck ve kalıcı volume'larla tanımlar.	Tüm backend servislerinin bağımlılığı; PostgreSQL ↔ Prisma, MinIO ↔ StorageService (ileride).
pgadmin/servers.json	Altyapı	DB yönetimi — pgAdmin'e 'HSE Docker' bağlantısını önceden tanımlı getirir.	pgAdmin → PostgreSQL
backend/package.json	Altyapı	Backend bağımlılık/script yönetimi — NestJS, Prisma, JWT, class-validator; prisma:migrate/seed scriptleri.	Tüm backend modüllerinin temeli.
backend/prisma/schema.prisma	Veri Modeli	Veri modelinin tek kaynağı — bu sprintte datasource/generator ve ilk model iskeleti; sonraki sprintlerde büyür.	PrismaService bu şemadan tip üretir; PostgreSQL'e migrate edilir.
backend/src/prisma/prisma.module.ts + prisma.service.ts	Backend	Veri erişim sarmalayıcı — PrismaClient'ı DI ile tüm modüllere sağlar (global).	app.module → tüm servisler → PrismaService → PostgreSQL
backend/src/app.module.ts	Backend	Kök modül — tüm modülleri ve global guard'ları (ileride) toplar.	main.ts tarafından bootstrap edilir; PrismaModule'ü içe alır.
backend/src/main.ts	Backend	Uygulama başlangıcı — global '/api' öneki, CORS, whitelist'li ValidationPipe.	AppModule'ü ayağa kaldırır; port 3000.
client/package.json + vite.config.ts	Altyapı	İstemci bağımlılık/derleme — React, Router, TanStack Query, axios, i18next; /api proxy.	Vite dev sunucusu /api → backend:3000.
client/src/main.tsx	Frontend	İstemci giriş noktası — React kökü, Router, QueryClient ve (ileride) AuthProvider'ı sarar.	App.tsx'i render eder.
client/src/App.tsx	Frontend	Rota iskeleti — kimlik durumuna göre rota ağacı (bu sprintte iskelet).	main.tsx → App → sayfalar (ileride).
client/src/styles.css	Frontend	Global stil — uygulama kabuğu, tipografi, renk paleti.	Tüm bileşenlerin ortak görünümü.
README.md	.gitignore ve dokümantasyon	Kurulum rehberi — hızlı başlangıç, ortam değişkenleri, varsayılan hesaplar.	Geliştirici onboarding.

3. Dosya İlişkileri (Topolojik Anlatım)

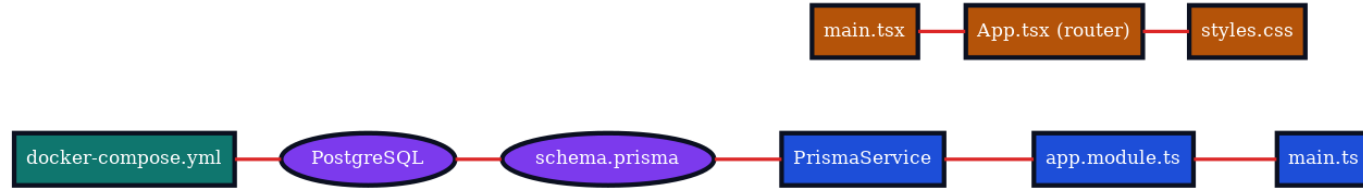
Bu sprintte ilişki grafiği en yalın halidir: docker-compose PostgreSQL'i sağlar, schema.prisma bu veritabanının şemasını tanımlar, PrismaService şemadan üretilen istemciyi sunar ve app.module bunu main.ts üzerinden ayağa kaldırır. İstemci tarafında main.tsx → App.tsx → styles.css zinciri kurulur. Henüz iki taraf birbirine API ile bağlanmamıştır; köprü (axios) bir sonraki sprintte gelir.

4. Kümülatif Entity-Relation Haritası (Cobweb)

Aşağıdaki harita, projenin başından bu sprintin sonuna kadar (Sprint 1...1) oluşan TÜM dosya ve veri düğümlerini ve aralarındaki ilişkileri gösterir. Bu sprintte eklenen düğümler kalın koyu çerçeve, yeni ilişkiler kırmızı kenarlarla

vurgulanmıřtır. Harita her sprintte büyüyerek örümcek ağını (cobweb) yoğunlařtırır: bu sprint sonunda 9 düğüm ve 7 kenar içerir.

Sprint 1 - Temel Altyapi & Sema | Kumulatif Entity-Relation Cobweb (Sprint 1..1)



Şekil — Sprint 1...1 kümülatif entity-relation cobweb haritası (9 düğüm / 7 kenar). Renkler: teal=Altyapi, mor=Veri Modeli, mavi=Backend, amber=Frontend, kırmızı=Cross-cutting.

Renk lejanti: ● Altyapi ● Veri Modeli ● Backend ● Frontend ● Cross-cutting

5. Sprint Çıktısı & Kabul Kriterleri

Sprint sonunda aşağıdaki demo-edilebilir çıktılar elde edilir:

- İki proje de hatasız derlenir ve ayağa kalkar (backend :3000/api, client :5173).
- docker compose up ile PostgreSQL + pgAdmin + MinIO çalışır.
- Prisma bağlantısı doğrulanır (prisma studio ile boş şema görülür).

Bir sonraki adım: Kimlik Doğrulama & RBAC Çekirdeği (Sprint 2) — cobweb 25 düğüme büyür.